

Fest-A 최종 발표 예상 Q&A

발표 직후 질문 대응용 | 기획, 기술, AI, 운영, 한계 및 개선 방향

발표 핵심 메시지

Fest-A 는 기존 축제 운영에서 흩어져 있던 방문객 정보, 운영진 소통, 관리자 도구를 하나의 PWA 플랫폼으로 통합하고, 실시간 혼잡도 확인, 30 분 후 AI 혼잡도 예측, 근거 기반 챗봇, 예약 및 QR 체크인, 관리자 운영 보조까지 연결한 대학 축제 특화 서비스입니다.

1. 먼저 말할 핵심 방어 포인트

- 문제정의: 방문객 정보 부족, 운영진 소통 지연, 통합 운영 도구 부재를 하나의 플랫폼 문제로 정의했습니다.
- 차별점: 단순 축제 소개 페이지가 아니라 혼잡도, 예약, QR 체크인, 실시간 공지, AI 챗봇, 관리자 운영 보조를 통합했습니다.
- AI 범위: RandomForest 기반 30 분 후 혼잡도 예측과 근거 기반 AI 챗봇을 구현했고, 모델 실패 시 fallback 로직으로 서비스가 멈추지 않게 했습니다.
- 기술 구조: React + Vite PWA 프론트, Spring Boot API, JPA 기반 DB, SSE 실시간 반영, OpenAI API 및 Python ML 모델 연동 구조입니다.
- 현실성: 전용 하드웨어를 전제로 하지 않고 GPS 로그, 예약, 체크인, 운영자 입력, 공연 맥락 같은 앱 기반 데이터를 활용했습니다.
- 한계 인정: 현재 AI 학습 데이터는 실제 장기간 운영 로그가 아니라 운영 가정과 시뮬레이션을 결합한 데이터입니다. 그래서 발표에서는 프로토타입과 확장 가능성으로 설명하는 것이 안전합니다.

2. 압박 질문에 대한 짧은 답변

질문 유형	핵심 답변	말할 때 주의점
-------	-------	----------

AI 진짜 구현?	네. RandomForest 모델 파일을 Python 추론 스크립트로 호출하고 API 응답에 반영합니다.	실제 장기 운영 데이터 검증은 향후 과제로 분리
정확도 충분?	RandomForest accuracy 0.7984, macro F1 0.79 로 프로토타입 기준 의미 있는 성능입니다.	안전 운영을 위해 fallback 과 관리자 판단을 함께 사용
전용 HW 없이 가능?	초기 버전은 GPS, 예약, 체크인, 운영자 입력 데이터로 추정합니다.	카메라 기반 추정은 향후 확장 방향
왜 PWA?	현장 방문객이 설치 부담 없이 접속하고, 필요하면 홈 화면 앱처럼 쓸 수 있습니다.	네이티브 앱보다 배포와 접근성이 유리
보안/개인정보?	JWT 보호, 운영 권한 분리, 위치/사용자 데이터 최소 수집과 익명화 방향으로 설계했습니다.	실운영 전 보안 점검은 필수

3. 기획 및 서비스 질문

Q. Fest-A 가 해결하려는 가장 본질적인 문제는 무엇인가요?

A. 축제 정보가 흩어져 있어 방문객은 현장 상황을 늦게 알고, 운영진은 카톡이나 구두 전달에 의존해 대응이 늦어지는 문제입니다. Fest-A 는 방문객, 부스 운영자, 스태프, 관리자가 같은 운영 데이터를 보도록 연결해 정보 단절을 줄이는 것을 목표로 했습니다.

Q. 단순한 축제 안내 앱과 무엇이 다른가요?

A. 일정과 부스 정보를 보여주는 데서 끝나지 않고, 혼잡도 확인, 30 분 후 혼잡 예측, 예약과 QR 체크인, 실시간 공지, 분실물, AI 챗봇, 관리자 대시보드까지 운영 흐름을 하나로 묶었습니다. 그래서 방문객 안내와 운영 관리가 동시에 가능한 구조입니다.

Q. 왜 대학 축제에 특화했다고 말할 수 있나요?

A. 대학 축제는 단기간 행사, 분산 부스, 비전문 운영진, 빠른 현장 변경이라는 특징이 있습니다. Fest-A는 부스 상태 변경, 대기/예약, 공연 임박, 스태프 알림, 관리자 공지 같은 현장 운영 항목을 중심으로 설계했기 때문에 일반 행사 앱보다 대학 축제 운영에 맞춰져 있습니다.

Q. 사용자가 이 앱을 실제로 쓸 이유는 무엇인가요?

A. 현장에서 바로 필요한 정보가 있기 때문입니다. 어느 부스가 덜 혼잡한지, 공연이 언제 시작하는지, 예약 상태가 어떤지, 분실물이나 공지가 있는지, 지금 어디로 가면 좋은지를 한 화면 흐름에서 확인할 수 있습니다.

Q. 운영진 입장에서 얻는 이점은 무엇인가요?

A. 운영진은 혼잡 구간, 예약 현황, 공지 반영, 부스 상태, 분실물 현황을 대시보드에서 확인할 수 있습니다. 특히 상태 변경이 SSE로 바로 반영되기 때문에 전화나 메신저로 반복 확인하는 시간을 줄일 수 있습니다.

Q. 기획 단계에서 기존 축제 운영의 어떤 한계를 보고 시작했나요?

A. 방문객 정보 부족, 운영진 소통 지연, 통합 운영 도구 부재가 핵심 문제였습니다. 부스 혼잡도나 품절 여부는 현장에 가기 전까지 알기 어렵고, 운영 정보는 카톡이나 구두 전달에 흩어져 누락과 지연이 생기기 쉽다고 판단했습니다.

Q. 차별화 포인트 6개 중 가장 중요한 3개만 꼽으면 무엇인가요?

A. 첫째는 앱 데이터 기반 혼잡도 추정, 둘째는 RandomForest 기반 30분 후 AI 혼잡도 예측, 셋째는 축제 데이터 기반 AI 챗봇입니다. 여기에 운영자/관리자 실시간 반영과 예약/QR 체크인을 보강 요소로 설명하면 됩니다.

Q. 부스 예약과 QR 체크인은 왜 필요한가요?

A. 인기 부스의 대기 부담을 낮추고, 운영자에게 실제 예약 및 입장 기록을 남기기 위해서입니다. 방문객은 대기 불확실성을 줄이고, 운영자는 예약 완료와 체크인을 구분해 현장 처리 상태를 관리할 수 있습니다.

4. UX 및 현장 적용 질문

Q. 축제 현장처럼 정신없는 환경에서 앱 사용성이 충분한가요?

A. 모바일 PWA 기반으로 주요 동선을 짧게 구성했습니다. 홈, 지도, 부스 상세, 예약, 챗봇, 더보기 같은 핵심 기능을 하단 탭 중심으로 접근하게 하고, 혼잡도와 상태는 색상과 라벨로 빠르게 읽히게 설계했습니다.

Q. 인터넷이 불안정한 현장에서는 어떻게 하나요?

A. PWA 구조라 기본 앱 셀, manifest, 오프라인 페이지, 일부 정적 리소스를 서비스 워커가 캐시합니다. 다만 실시간 혼잡도나 예약처럼 최신성이 중요한 기능은 네트워크 연결이 필요하며, 연결 장애 시 오프라인 안내와 fallback 안내로 사용자를 막지 않도록 설계했습니다.

Q. 방문객이 앱 설치를 귀찮아하지 않을까요?

A. 네이티브 앱 설치가 아니라 웹 접속 기반 PWA 라 QR 코드나 링크로 바로 접근할 수 있습니다. 필요하면 홈 화면에 추가할 수 있지만, 기본 이용은 브라우저에서 가능합니다.

Q. QR 을 찍고 다시 방문하는 흐름은 어떻게 보장하나요?

A. QR 이나 링크로 접속했을 때 부스 상세, 예약, 체크인 흐름으로 바로 들어오게 만들 수 있습니다. 향후에는 전용 도메인과 QR 랜딩 구조를 강화해 재방문 동선을 더 짧게 만들 계획입니다.

Q. 외국인이나 접근성 사용자는 고려했나요?

A. 현재 구조는 모바일 가독성과 명확한 상태 라벨에 집중했습니다. 향후 개선 방향에는 접근성 강화와 서비스 완성도 개선이 포함되어 있으며, 실제 운영 전에는 색상 대비, 버튼 크기, 다국어 안내, 스크린 리더 대응을 추가 점검해야 합니다.

Q. 사용자가 많아지면 화면이 복잡해지지 않나요?

A. 방문객 화면과 관리자 화면을 분리했고, 방문객에게는 현재 필요한 정보만 노출합니다. 상세한 운영 로그와 KPI 는 관리자 대시보드로 보내 방문객 화면이 과도하게 복잡해지지 않게 했습니다.

5. 기술 아키텍처 질문

Q. 전체 시스템 구조를 한 문장으로 설명하면?

A. React + Vite PWA 프론트가 Spring Boot API 와 통신하고, 백엔드는 JPA 기반 DB, SSE 실시간 스트림, OpenAI API, Python ML 모델을 연결해 방문객과 운영자 화면에 데이터를 제공합니다.

Q. 왜 React 와 Vite 를 선택했나요?

A. React 는 컴포넌트 기반으로 방문객, 관리자, 운영자 화면을 재사용하기 좋고, Vite 는 개발 서버와 빌드 속도가 빠릅니다. PWA 와 정적 배포 구조에도 잘 맞아 Vercel 배포가 단순합니다.

Q. 왜 백엔드는 Spring Boot 인가요?

A. 축제 운영 서비스는 인증, 관리자 API 보호, 데이터 CRUD, 예약 상태 처리, SSE 이벤트 발행처럼 서버 책임이 많습니다. Spring Boot 는 JPA, Security, Validation, REST API 구성이 안정적이라 운영 관리형 서비스에 적합하다고 판단했습니다.

Q. 실시간 반영은 어떻게 구현했나요?

A. SSE(Server-Sent Events)를 사용했습니다. 프론트는 EventSource 로 구독하고, 백엔드는 혼잡도, 공연, 공지, 부스, 스태프, 분실물, 예약 변경 시 관련 이벤트를 발행해 새로고침 없이 화면에 반영합니다.

꼬리질문 대응: 양방향 채팅이 핵심이 아니라 서버 변경사항을 클라이언트에 밀어주는 흐름이므로 WebSocket 보다 SSE 가 단순하고 충분합니다.

Q. DB 는 MySQL 인가요 PostgreSQL 인가요?

A. 로컬 개발 기본값은 MySQL 로 잡혀 있고, 발표 아키텍처의 배포 구조는 Railway 의 PostgreSQL 구성을 기준으로 설명했습니다. 백엔드는 Spring Data JPA 와 JDBC 드라이버 구조라 환경변수로 DB 연결을 바꿀 수 있습니다.

꼬리질문 대응: 질문이 나오면 '개발 편의상 로컬은 MySQL, 배포는 PostgreSQL 도 가능한 구조'라고 짧게 답하면 안전합니다.

Q. 관리자 API 보안은 어떻게 처리했나요?

A. 관리자 로그인 후 JWT 를 발급하고, `/api/admin/\*\*` 같은 관리자 API 는 인증 필터와 Spring Security 설정으로 보호합니다. 운영 콘솔에는 별도 운영 키와 권한 분리를 뒤 방문객 기능과 관리 기능을 분리했습니다.

Q. 외부 API 장애가 나면 서비스가 멈추나요?

A. 그렇게 설계하지 않았습니다. OpenAI 응답이 실패하면 근거 기반 기본 안내로 fallback 하고, Python ML 모델 실행이 실패하면 규칙 기반 fallback 으로 혼잡도 결과를 생성합니다. 외부 의존 기능이 실패해도 핵심 조회와 운영 기능은 유지되게 했습니다.

Q. 프론트와 백엔드 배포는 어떻게 나뉘나요?

A. 발표자료 기준으로 프론트는 Vercel 의 정적 PWA 배포, 백엔드는 Railway 의 Spring Boot API 배포, DB 는 Railway PostgreSQL 플러그인 구조입니다. 프론트는 HTTPS 로 API 를 호출하고, 실시간 수신은 SSE 연결로 처리합니다.

6. AI 혼잡도 예측 질문

Q. AI 혼잡도 예측은 정확히 무엇을 예측하나요?

A. 현재 축제 상황 데이터를 24 개 feature 로 변환해 30 분 후 부스별 혼잡 단계를 예측합니다. 출력은 LOW, NORMAL, BUSY, VERY_BUSY 같은 등급과 위험 점수, 추천 부스 또는 회피 부스 형태로 화면에 표시됩니다.

Q. 입력 feature 에는 어떤 것들이 있나요?

A. 시간 정보, 공연 정보, 공연 임박 신호, 위치/부스 정보, GPS 혼잡 신호, 예약/체크인, 대기 시간/재고 데이터가 포함됩니다. 예를 들면 hour, is_peak_time, artist_popularity_score, stage_load_ratio, gps_count_nearby, reservation_count, checked_in_count, wait_minutes 같은 값입니다.

Q. 왜 30 분 후를 예측하나요?

A. 방문객이 지금 이동할지, 다른 부스를 먼저 갈지 결정하기에 30 분은 현실적인 의사결정 단위입니다. 운영자도 30 분 단위면 공지, 인력 이동, 부스 상태 변경 같은 대응을 할 수 있습니다.

Q. 왜 RandomForest 를 운영 모델로 선택했나요?

A. XGBoost 가 성능은 조금 높았지만 차이가 크지 않았고, RandomForest 는 scikit-learn 기반 저장과 Python 추론 연동이 단순하며 feature importance 로 설명하기 쉽습니다. 실시간 운영에서는 최고 성능 1 개보다 안정성과 설명 가능성이 중요하다고 봤습니다.

Q. 모델 성능은 어느 정도인가요?

A. 발표자료 기준 RandomForest 는 accuracy 0.7984, macro F1 0.79 이고, XGBoost 는 accuracy 0.8143, macro F1 0.81 입니다. 차이가 크지 않아 운영 모델은 RandomForest 로 두고 XGBoost 는 비교 실험 모델로 제시했습니다.

Q. 학습 데이터는 실제 데이터인가요?

A. 현재는 HYBRID_SIMULATED 로, 실제 운영 가정과 시뮬레이션을 결합한 학습 데이터입니다. 실제 축제 운영 로그가 충분히 쌓이면 예약, 체크인, GPS, 대기시간, 운영자 입력 데이터를 기반으로 재학습하는 것이 다음 단계입니다.

꼬리질문 대응: 이 질문에는 과장하지 말고 '실제 장기 운영 검증 모델은 아직 아니며, 구조와 프로토타입을 구현했다'고 답하는 것이 좋습니다.

Q. AI 예측이 틀리면 현장 운영에 위험하지 않나요?

A. 예측값을 자동 의사결정으로 쓰는 것이 아니라 방문객 추천과 운영자 보조 지표로 사용합니다. 또한 fallback 규칙과 관리자 판단을 함께 두어 모델 오류가 바로 운영 장애로 이어지지 않도록 했습니다.

Q. 모델 실패 시 fallback 은 어떻게 동작하나요?

A. Python 모델 실행이 실패하거나 입력 데이터가 부족하면 최근 GPS, 예약, 체크인, 대기시간, 부스 상태 같은 사용 가능한 정보로 규칙 기반 점수를 계산합니다. 일부 스냅샷 데이터가 비어 있어도 가능한 정보만 반영해 결과를 만듭니다.

Q. Feature importance 는 어떤 의미가 있나요?

A. 어떤 입력값이 혼잡도 예측에 크게 영향을 줬는지 설명하는 근거입니다. 운영자에게 '왜 이 부스를 회피하라고 했는지', '왜 이 구간을 위험하게 봤는지' 설명할 수 있어 단순 블랙박스 예측보다 운영 설득력이 있습니다.

Q. 향후 AI 고도화는 어떤 방향인가요?

A. 실제 운영 데이터 기반 재학습, 시간 흐름을 반영하는 시계열 모델, drift 감지와 자동 재학습, SHAP 기반 설명 가능 AI, 사용자 위치와 선호를 반영한 개인화 동선 추천으로 확장할 수 있습니다.

7. AI 챗봇 및 AI Match 질문

Q. 챗봇은 그냥 OpenAI 에 질문을 보내는 구조인가요?

A. 아닙니다. 사용자 질문을 받은 뒤 부스, 공연, 공지, 분실물, 혼잡도 같은 축제 데이터를 먼저 찾고, 그 근거를 바탕으로 답변을 구성합니다. OpenAI 응답을 쓰더라도 축제 데이터 기반 근거를 함께 제공하는 구조입니다.

Q. 챗봇이 틀린 답을 하면 어떻게 하나요?

A. 관련 정보가 부족하면 일반 안내나 대체 답변을 제공하고, AI 응답 생성이 원활하지 않으면 근거 기반 fallback 답변을 사용합니다. 발표에서는 '정확한 운영 데이터 조회 + 자연어 설명 보조'로 보는 것이 적절합니다.

Q. AI 챗봇의 입력과 출력 데이터는 무엇인가요?

A. 입력은 사용자 질문과 축제 데이터입니다. 축제 데이터에는 부스 정보, 공연 일정, 공지사항, 분실물 정보, 혼잡도 정보가 포함되고, 출력은 최종 답변, 관련 근거 정보, 추가 안내 또는 주의 메시지입니다.

Q. AI Match 는 발표에서 어떤 의미인가요?

A. 실제 적용 사례로, AI 기반 매칭 기능을 Fest-A 플랫폼 위에 확장 적용한 예시입니다. 사용자 프로필과 신청 데이터를 바탕으로 매칭과 운영 관리를 지원하고, 관리자 화면에서 신청/매칭 현황을 확인할 수 있게 구성했습니다.

Q. AI Match 는 본 서비스 핵심인가요, 부가 기능인가요?

A. 핵심은 축제 통합 운영 플랫폼이고, AI Match 는 그 플랫폼이 부스/공연/공지뿐 아니라 축제 이벤트형 서비스까지 확장될 수 있음을 보여주는 실제 적용 사례로 설명하면 됩니다.

8. 데이터, 보안, 개인정보 질문

Q. GPS 데이터를 쓰면 개인정보 문제가 생기지 않나요?

A. 실운영에서는 위치 데이터 최소 수집, 익명화, 보관 기간 제한, 목적 고지와 동의가 필요합니다. 현재 설계는 개별 사용자를 추적하기보다 구역 단위 혼잡 신호로 활용하는 방향이며, 향후 보안 강화 항목에 위치/사용자 데이터 익명화와 최소 수집을 포함했습니다.

Q. SMS 인증은 왜 필요한가요?

A. 예약이나 AI Match 처럼 사용자의 중복 신청, 허위 신청, 체크인 혼선을 줄여야 하는 기능에서 필요합니다. 다만 SMS provider 는 환경변수로 설정하고, 로컬 개발에서는 Noop 또는 테스트 모드로 동작하게 할 수 있습니다.

Q. 관리자 계정이 노출되면 위험하지 않나요?

A. 운영 환경에서는 기본 계정을 사용하지 않고 환경변수로 관리자 계정, JWT secret, 운영 키를 설정해야 합니다. 발표에서는 로컬 데모용 계정과 운영 배포 보안 설정을 분리했다고 설명하면 됩니다.

Q. 데이터가 많아지면 DB 성능은 괜찮나요?

A. 초기 축제 규모에서는 JPA 기반 CRUD 와 인덱스 설계로 충분히 대응 가능합니다. 실제 운영 데이터가 쌓이면 GPS 로그, 예약 로그, SSE 이벤트성 데이터는 조회 패턴에 맞춰 인덱스와 보관 정책을 분리해야 합니다.

Q. OpenAI API 에 개인정보가 넘어갈 수 있지 않나요?

A. 챗봇에는 축제 정보와 사용자 질문 위주로 전달하고, 민감한 개인정보는 보내지 않는 방향으로 설계해야 합니다. 실운영에서는 프롬프트 구성 단계에서 개인정보 제거, 로그 마스킹, API 사용 정책 검토가 필요합니다.

9. 운영 및 확장성 질문

Q. 축제 당일 동시 접속이 몰리면 어떻게 대응하나요?

A. 정적 프론트는 Vercel 같은 CDN 기반 배포로 확장성이 좋고, 백엔드는 API 와 SSE 연결 부하를 별도로 관리해야 합니다. 향후 개선 방향에 서버 부하 테스트와 장애 시 대체 안내 페이지를 포함했습니다.

Q. SSE 연결이 많아지면 서버에 부담이 되지 않나요?

A. SSE 는 단방향 실시간 알림에는 단순하고 효율적이지만, 접속자가 크게 늘면 연결 수 관리가 필요합니다. 큰 규모 운영에서는 채널 분리, 이벤트 범위 축소, 캐시, 메시지 브로커 도입을 검토할 수 있습니다.

Q. 운영자가 잘못된 정보를 입력하면 어떻게 하나요?

A. 현재는 관리자와 운영자 권한을 분리하고, 운영 로그와 감사 로그로 변경 이력을 확인할 수 있게 구성했습니다. 실운영에서는 중요 공지 승인 절차, 변경 취소, 입력 검증을 강화하면 됩니다.

Q. 다른 학교나 다른 행사에도 적용 가능한가요?

A. 가능합니다. 부스, 공연, 공지, 예약, 지도, 관리자 운영이라는 구조는 다른 대학 축제나 지역 행사에도 적용할 수 있습니다. 다만 축제별 지도, 운영 권한, 데이터 수집 정책, 부스 유형은 설정화해야 합니다.

Q. 실제 운영 전 반드시 해야 할 테스트는 무엇인가요?

A. 예약과 QR 체크인 반복 테스트, 관리자 공지 반영 테스트, SSE 실시간 반영 테스트, 모바일 네트워크 불안정 상황 테스트, 동시 접속 부하 테스트, 개인정보 동의 및 보안 점검이 필요합니다.

Q. 개발 완료 현황에서 고도화 예정으로 남은 것은 무엇인가요?

A. 핵심 서비스 기능과 AI 기능은 구현되어 있고, 실제 운영 데이터 기반 재학습, 설명 가능한 AI 와 drift 감지, 개인화 추천 확장은 고도화 예정으로 분류했습니다. 즉, 현재는 시연 가능한 통합 플랫폼이고, 운영 데이터 축적 이후 AI 를 더 정교하게 만드는 단계가 남아 있습니다.

10. 한계와 개선 방향 질문

Q. 현재 서비스의 가장 큰 한계는 무엇인가요?

A. 실제 축제 장기간 운영 로그로 검증된 AI 모델이 아니라는 점입니다. 그래서 현재 모델은 구조와 가능성을 보여주는 프로토타입이고, 실제 운영 후 데이터가 쌓이면 재학습과 정확도 검증을 해야 합니다.

Q. 기획적으로 가장 아쉬운 부분은 무엇인가요?

A. 실제 축제 현장에서 충분한 사용자 테스트를 하지 못했다는 점입니다. 향후에는 방문객 동선, QR 접속률, 부스 운영자 입력 부담, 관리자 대응 시간을 측정해 기능 우선순위를 조정해야 합니다.

Q. 기술적으로 가장 개선하고 싶은 부분은 무엇인가요?

A. 부하 테스트와 장애 대응, 데이터 익명화, AI 모델 재학습 파이프라인, 관측 가능성 로그를 강화하고 싶습니다. 특히 축제 당일에는 예측 정확도뿐 아니라 서비스 안정성이 더 중요하기 때문입니다.

Q. 카메라 기반 인원 추정은 왜 현재 넣지 않았나요?

A. 카메라는 정확도 향상 가능성이 있지만 설치 위치, 개인정보, 운영 비용, 학교 승인 문제가 있습니다. 초기 버전은 전용 하드웨어 없이 앱 데이터로 시작하고, 필요할 때 카메라 기반 인원 추정을 검토하는 방향이 현실적입니다.

Q. 향후 개선 방향을 5 개만 말하면?

A. 데이터 수집 방식 확장, 서비스 안정성 및 보안 강화, 지속적인 QA 와 오류 개선, 축제 현장에 적합한 UI/UX 개선, 접근성 및 서비스 완성도 강화입니다. 발표자료의 향후 개선 방향도 이 5 가지로 정리되어 있습니다.

Q. 이 프로젝트를 한 문장으로 마무리하면?

A. Fest-A 는 대학 축제 현장의 정보 단절을 줄이기 위해 방문객 안내, 운영 관리, 실시간 반영, AI 예측과 챗봇을 하나의 PWA 기반 플랫폼으로 연결한 서비스입니다.

11. 발표 중 답변 톤 가이드

- AI 질문에는 과장하지 말고 '현재는 구현된 프로토타입, 실제 운영 데이터 축적 후 재학습'이라고 답합니다.
- 정확도 질문에는 수치와 함께 '운영 보조 지표이며 fallback 과 관리자 판단을 병행한다'고 말합니다.
- 개인정보 질문에는 '최소 수집, 익명화, 보관 기간 제한, 실운영 전 고지와 동의'를 빠짐없이 언급합니다.
- DB 나 배포 구조 질문에는 '환경변수 기반으로 연결 DB 를 바꿀 수 있고, 발표 배포 구조는 PostgreSQL 기준'이라고 정리합니다.
- 한계 질문에는 숨기지 말고 실제 현장 검증, 부하 테스트, 재학습 파이프라인을 향후 과제로 제시합니다.

12. 참고한 발표자료 흐름

- 기획 출발점: 방문객 정보 부족, 운영진 소통 지연, 통합 운영 도구 부재
- 차별점: SW 기반 혼잡도 추정, AI 혼잡도 예측, 축제 데이터 기반 AI 챗봇, 대학 축제 특화 설계, 분산형 운영 관리, 예약/QR 체크인
- 기술 구조: React + Vite PWA, Spring Boot API, JPA DB, SSE, OpenAI API, Python ML 모델
- AI 혼잡도: 24 개 feature, 30 분 후 예측, RandomForest 운영 모델, XGBoost 비교 실험, fallback 보정
- 고도화: 실제 데이터 재학습, 시계열 모델, drift 감지, SHAP, 개인화 추천